VCGen

SeaHorn Style

Idea

SeaHorn-style VCGen keeps control flow explicit.

For each block `i`, introduce a Boolean reachability variable:

$$\mathtt{bb}_i$$

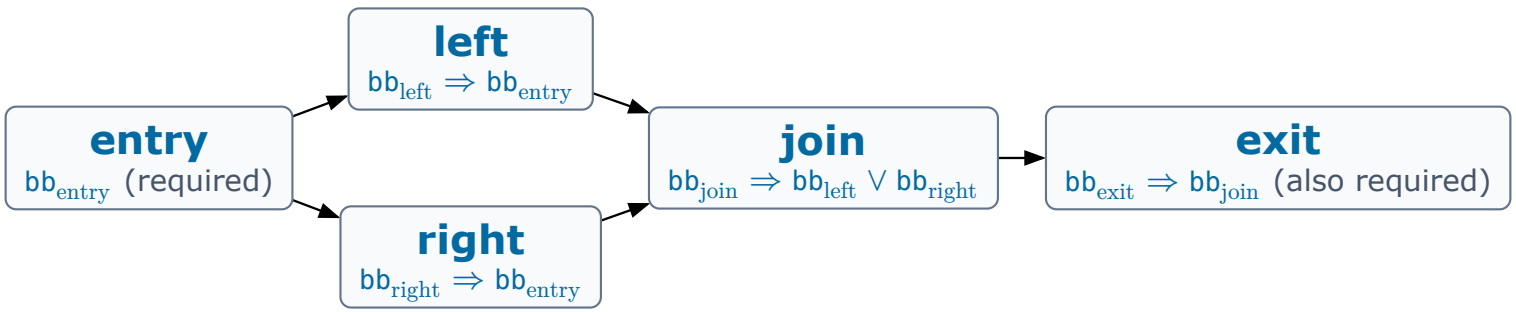
Then constrain the selected blocks so they describe an entry-to-exit path.

$$\mathrm{VCGen\_SeaHorn}(P) = \mathbf{CFG} \wedge \mathbf{DEF} \wedge \mathbf{JUMPS} \wedge \mathbf{PHI} \wedge \mathbf{ERROR}$$

- `CFG`: selected block variables form a path.
- `DEF`: assignments hold when their block is selected.
- `JUMPS`: selected branch edges satisfy branch conditions.
- `PHI`: selected incoming edges choose phi values.
- `ERROR`: the selected exit violates the assertion.

The Diamond, Encoded On The Picture

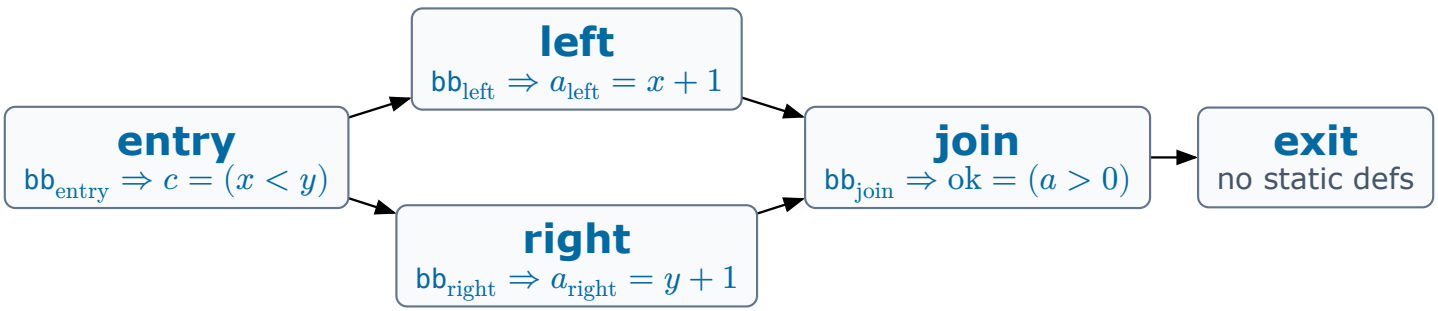
Each component lives somewhere on the graph:



CFG — every selected block has a selected predecessor; `entry` and `exit` are forced.

The Diamond, Encoded On The Picture

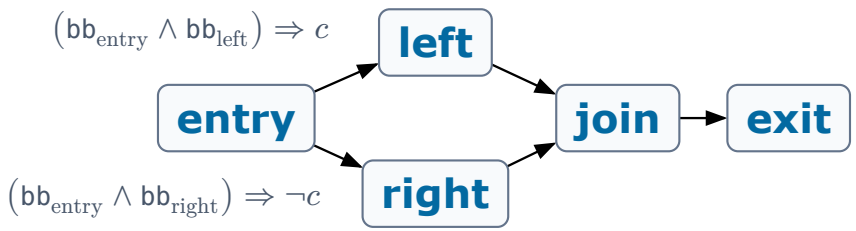
Each component lives somewhere on the graph:



DEF — assignments become equalities, guarded by their block. Havoc contributes nothing.

The Diamond, Encoded On The Picture

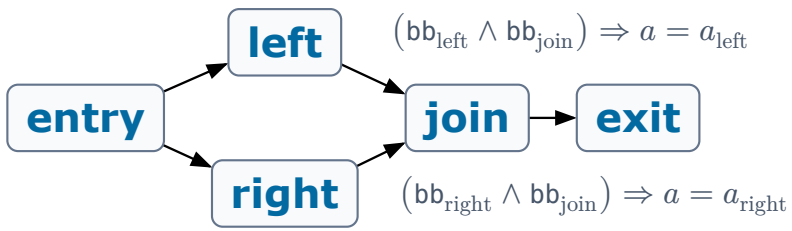
Each component lives somewhere on the graph:



JUMPS — branch conditions attach to the **edges** of the conditional terminator.

The Diamond, Encoded On The Picture

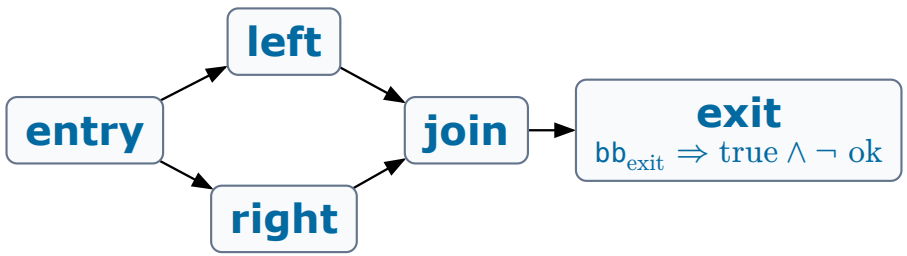
Each component lives somewhere on the graph:



PHI — the selected incoming edge chooses the phi value. (The DSA idea, expressed on edges.)

The Diamond, Encoded On The Picture

Each component lives somewhere on the graph:



ERROR — the selected exit must satisfy `pre` and falsify `post`. With $\mathtt{bb}_{\mathrm{exit}}$ forced, a model is a complete failing execution.

The Whole Formula

$$
\begin{aligned}
\textbf{CFG} &= \mathtt{bb}_{\text{entry}} \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow \mathtt{bb}_{\text{entry}}\right) \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow \mathtt{bb}_{\text{entry}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}\right) \wedge \left(\mathtt{bb}_{\text{exit}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \wedge \mathtt{bb}_{\text{exit}} \\
\textbf{DEF} &= \left(\mathtt{bb}_{\text{entry}} \Rightarrow c = (x < y)\right) \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow a_{\text{right}} = y + 1\right) \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \mathrm{ok} = (a > 0)\right) \\
\textbf{JUMPS} &= \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}}\right) \Rightarrow c\right) \wedge \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{right}}\right) \Rightarrow \neg c\right) \\
\textbf{PHI} &= \left(\left(\mathtt{bb}_{\text{left}} \wedge \mathtt{bb}_{\text{join}}\right) \Rightarrow a = a_{\text{left}}\right) \wedge \left(\left(\mathtt{bb}_{\text{right}} \wedge \mathtt{bb}_{\text{join}}\right) \Rightarrow a = a_{\text{right}}\right) \\
\textbf{ERROR} &= \mathtt{bb}_{\text{exit}} \Rightarrow \top \wedge \neg\, \mathrm{ok}
\end{aligned}
$$

A satisfying model is a candidate unsafe execution: a selected entry-to-exit path, consistent assignments and edge conditions, and a false assertion at `exit`.

Optimization: Unguarded DEF

Static definitions may be exposed as top-level equations:

Guarded:

$$\mathsf{bb}_i \Rightarrow x = y + 1$$

Unguarded:

$$x = y + 1$$

Optimization: Unguarded DEF

Static definitions may be exposed as top-level equations:

Guarded:

$$\mathtt{bb}_i \Rightarrow x = y + 1$$

Unguarded:

$$x = y + 1$$

Why bother? Top-level equalities feed algebraic simplification:

$$x = y + 1 \land z = x + 2 \quad \Longrightarrow \quad z = y + 3$$

Sound only when the definition and its side conditions are globally safe — **keep that clause in mind for the pitfalls.**

Optimization: Phi As ITE

Edge implications:

$$(\mathsf{bb}_i \wedge \mathsf{bb}_j) \Rightarrow x = x_i$$

$$(\mathsf{bb}_k \wedge \mathsf{bb}_j) \Rightarrow x = x_k$$

One defining equation:

$$x = \mathrm{ite}(\mathsf{bb}_i, x_i, x_k)$$

After Boolean propagation learns bb_i, the ITE collapses to $x = x_i$.

The ITE form gives the merge a syntactic definition the solver can substitute — at a price we will meet in pitfall 2.

The Optimized Diamond

Same program — forward CFG, unguarded DEF, phi as ITE:

$$
\begin{aligned}
\textbf{CFG} &= \mathtt{bb}_{\text{entry}} \wedge \left(\mathtt{bb}_{\text{entry}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}\right) \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \mathtt{bb}_{\text{exit}}\right) \wedge \mathtt{bb}_{\text{exit}} \\
\textbf{DEF} &= c = (x < y) \wedge a_{\text{left}} = x + 1 \wedge a_{\text{right}} = y + 1 \\
&\quad \wedge a = \mathrm{ite}\left(\mathtt{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}}\right) \wedge \mathrm{ok} = (a > 0) \\
\textbf{JUMPS} &= \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}}\right) \Rightarrow c\right) \wedge \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{right}}\right) \Rightarrow \neg c\right) \\
\textbf{PHI} &= \top \\
\textbf{ERROR} &= \mathtt{bb}_{\text{exit}} \Rightarrow \top \wedge \neg\, \mathrm{ok}
\end{aligned}
$$

Reachability flows forward; definitions are naked equations ready for substitution; the phi component is **empty** — the merge lives in DEF as an ITE.

Pitfall 1: A Critical Edge

```
entry:
  c := havoc
  if c goto join else mid

mid:
  goto join

join:
  ok := phi [entry: true, mid: false]
```

The encoder emits the true-edge condition:

$$\left(\mathsf{bb}_{\text{entry}} \wedge \mathsf{bb}_{\text{join}}\right) \Rightarrow c$$

The program is unsafe (take the `mid` path). Does the encoder find the bug?

Pitfall 1: A Critical Edge

```
entry:
  c := havoc
  if c goto join else mid

mid:
  goto join

join:
  ok := phi [entry: true, mid: false]
```

The encoder emits the true-edge condition:

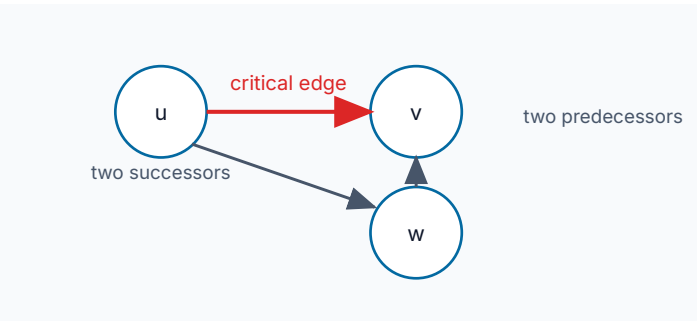
$$\left(\mathsf{bb}_{\mathrm{entry}} \wedge \mathsf{bb}_{\mathrm{join}}\right) \Rightarrow c$$

The program is unsafe (take the `mid` path). Does the encoder find the bug?

No. On `entry -> mid -> join`, both $\mathsf{bb}_{\mathrm{entry}}$ and $\mathsf{bb}_{\mathrm{join}}$ are true while c is false — the constraint **rejects the real bug path**. Block variables cannot tell which incoming edge was taken.

Pitfall 1: The Repair — Split The Edge

An edge $u \to v$ is critical when $|\text{succ}(u)| > 1 \wedge |\text{pred}(v)| > 1$:



Insert a landing block:

```
entry:
  c := havoc
  if c goto e2j else mid
e2j:
  goto join
```

The condition attaches to a non-critical edge:

$$\left(\text{bb}_{\text{entry}} \wedge \text{bb}_{\text{e2j}}\right) \Rightarrow c$$

bb_{e2j} is true **only** on the true edge.

Pitfall 2: A Safe Program Goes SAT

```
left:
  x_left := true
  p_left := true
  goto join

right:
  x_right := false
  p_right := false
  goto join

join:
  x := phi [left: x_left, right: x_right]
  p := phi [left: p_left, right: p_right]
  ok := x == p
```

Safe: both paths make `x` and `p` equal.

The two phis are emitted as independent ITEs, cases in different orders:

$$x = \mathrm{ite}\left(\mathtt{bb}_{\mathrm{left}}, x_{\mathrm{left}}, x_{\mathrm{right}}\right)$$

$$p = \mathrm{ite}\left(\mathtt{bb}_{\mathrm{right}}, p_{\mathrm{right}}, p_{\mathrm{left}}\right)$$

The solver reports SAT. What model did it find?

Pitfall 2: A Safe Program Goes SAT

```
left:
  x_left := true
  p_left := true
  goto join

right:
  x_right := false
  p_right := false
  goto join

join:
  x := phi [left: x_left, right: x_right]
  p := phi [left: p_left, right: p_right]
  ok := x == p
```

Safe: both paths make `x` and `p` equal.

The two phis are emitted as independent ITEs, cases in different orders:

$$x = \mathrm{ite}\left(\mathtt{bb}_{\mathrm{left}}, x_{\mathrm{left}}, x_{\mathrm{right}}\right)$$

$$p = \mathrm{ite}\left(\mathtt{bb}_{\mathrm{right}}, p_{\mathrm{right}}, p_{\mathrm{left}}\right)$$

The solver reports SAT. What model did it find?

$\mathtt{bb}_{\mathrm{left}} = \mathtt{bb}_{\mathrm{right}} = \mathrm{true}$: the first ITE reads the **left** value, the second reads the **right** value — $x = \mathrm{true}, p = \mathrm{false}$. A mixed state no execution produces: a **spurious counterexample**.

Pitfall 2: The Repair — Predecessor Exclusivity

With edge implications, selecting both predecessors is an immediate contradiction ($x = x_{\text{left}}$ and $x = x_{\text{right}}$ conflict).

An ITE merge has only **one** equality — it silently picks an arm.

The repair is an at-most-one constraint at the merge:

$$\neg\left(\mathsf{bb}_{\text{left}} \wedge \mathsf{bb}_{\text{right}}\right)$$

Equivalently: any CFG encoding with edge variables must enforce that at most one incoming edge to a merge fires.

Phi-as-ITE trades a built-in exclusivity check for solver-friendliness — the side condition must come back explicitly.

Pitfall 3: An Unguarded Path-Local Fact

`narrow64` is identity, but its encoding adds a 64-bit range fact for its result:

```
entry:
  x := havoc
  small := x < 2^64
  if small goto narrow_block else exit

narrow_block:
  y := narrow64(x)
  goto exit

exit:
  assume not small
  assert false
  halt
```

Unsafe: pick $x \geq 2^{64}$, skip `narrow_block`, reach `assert false`.

The encoder emits the definition **unguarded**:

$$y = x \wedge 0 \leq y \wedge y \leq 2^{64} - 1$$

Does the solver find the bug?

Pitfall 3: An Unguarded Path-Local Fact

`narrow64` is identity, but its encoding adds a 64-bit range fact for its result:

```
entry:
  x := havoc
  small := x < 2^64
  if small goto narrow_block else exit

narrow_block:
  y := narrow64(x)
  goto exit

exit:
  assume not small
  assert false
  halt
```

Unsafe: pick $x \geq 2^{64}$, skip `narrow_block`, reach `assert false`.

The encoder emits the definition **unguarded**:

$$y = x \wedge 0 \leq y \wedge y \leq 2^{64} - 1$$

Does the solver find the bug?

No — UNSAT. The global range fact bounds x on **every** path; the failing path needs $x \geq 2^{64}$ and is now inconsistent. A fact from an unvisited block killed a real bug.

Pitfall 3: The Repair — Guard Path-Local Facts

Keep the range fact local to its block:

$$\texttt{bb}_{\text{narrow\_block}} \Rightarrow \left(y = x \wedge 0 \leq y \wedge y \leq 2^{64} - 1\right)$$

The unguarded-DEF optimization is safe for **total** right-hand sides. It breaks the moment the definition smuggles in a **path-local axiom** — a range fact, a partial-operation side condition, an operator axiom valid only where the call appears.

Guard such facts by their triggering block, or prove them globally safe.

The Two Ways To Be Wrong

Pitfall	Effect	Failure mode
critical edges	**missing paths** — a real execution is ruled out	bug silently missed (unsoundness)
ITE merge without exclusivity	**infeasible paths** — the formula admits states no execution produces	spurious counterexample (incompleteness)
unguarded path-local facts	**missing paths** — facts from an unvisited block constrain the failing path	bug silently missed (unsoundness)

Missing paths break **soundness**: the tool is silent on a real bug — a wrong "proved". Infeasible paths break **completeness**: noise — a false alarm on a safe program. Every encoding change should be audited against both directions.